

Go Assignment: Service→Logic Pattern with Dependency Injection

Objective

Build a **parcel delivery notification system** in Go using the service→logic pattern and dependency injection. The assignment mimics a real-world scenario of notifying a customer when a parcel's status changes (e.g., "shipped," "delivered," "delayed")—focusing on clean separation between **service and business logic** through interfaces.

Scenario Overview

A delivery company needs a **Notification Service** that alerts customers about parcel status updates via channels like email and SMS. The notification logic (who to notify, message content, etc.) should be separate from the communication methods (email, SMS services).

Assignment Requirements

1. Service Layer

- Implement a **NotificationService** struct.
- This service should have a dependency on an interface, e.g., **Notifier**.
- Use dependency injection to wire up dependencies at runtime.

2. Logic Layer

- Implement a **ParcelStatusNotifier** interface that describes how to notify about parcel status changes.
- Provide at least two implementations, e.g., **EmailNotifier** and **SMSNotifier**.
- Logic for formatting messages and handling notification rules (e.g., "do not notify outside 8am–8pm") should be in these implementations.

3. Application Flow

- Simulate a list of customers and parcels with different statuses as in-memory structs or slices.
- On status change, the service calls the notifier to deliver a notification.

4. Dependency Injection

- Accept **Notifier** implementations in the **NotificationService** via constructor injection.
- Easily swap notifiers (e.g., for testing or changing business rules).